# Empirical Analysis of Insertion Sort

Category: Insertion Sort / Sorting | Language: Python 3

## 1. Aim

To test how Insertion Sort behaves for different array sizes and input orders. I counted comparisons and shifts, measured execution time, and compared the results with the expected time complexity.

## 2. Implementation and method

The Python program moves each new key left until it reaches its correct position. One comparison is counted every time an existing array value is checked against the key; one shift is counted when an item moves one position to the right.

Input sizes were 100, 300, 600, 1000, 2000, and 5000 elements. For each size I used sorted, reverse-sorted, and random values. A fixed random seed (42) makes the test values repeatable. Timing used **time.perf_counter()**.

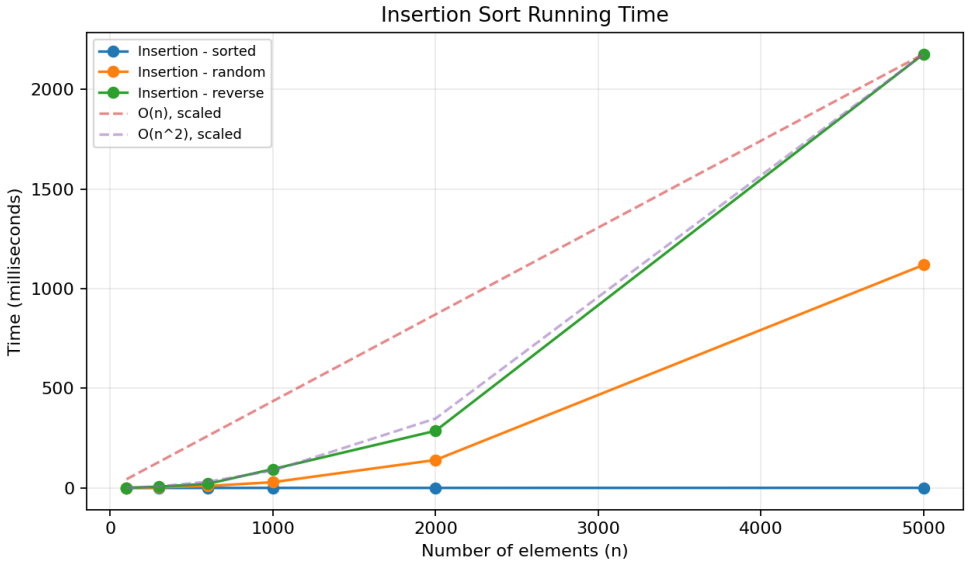
## 3. Selected experimental results

| Input, n=5000 | Time (ms) | Comparisons | Shifts |
|---|---|---|---|
| Sorted | 0.499 | 4999 | 0 |
| Random | 1119.068 | 6214859 | 6209871 |
| Reverse | 2177.369 | 12497500 | 12497372 |

## 4. Analysis

Sorted input needs almost no shifts, so its runtime grows approximately as **O(n)**. Reverse-sorted input needs about n(n-1)/2 comparisons and shifts when all values differ, which shows **O(n^2)** growth. Random input is usually between these two cases.

The following graph displays measured times and scaled reference curves. The theoretical curves are scaled to help compare shapes, not absolute speed.




## 5. Comparison with Merge Sort

Merge Sort divides the array into halves and takes **O(n log n)** time even for reverse-sorted input, while Insertion Sort may take **O(n^2)**. Insertion Sort is simpler and can be useful on very small or nearly sorted arrays. Merge Sort uses more extra memory for merging.